

Please read the instructions for Task 5.1.2 on the ICT1011 Lab 1 Manual first!!!

Lab 2 – CCStudio Projects

The objective of this lab is to learn the basic features of Code Composer Studio. In this exercise you will create a new project, build the code, and program the on-chip flash on the MSP430 device.

Lab 2 – Creating CCS Projects

◆ Lab 2a – Hello World

- Create a new project
- Build program, launch debugger, connect to target, and load your program
- printf() to CCS console

◆ Lab 2b – Blink the LED

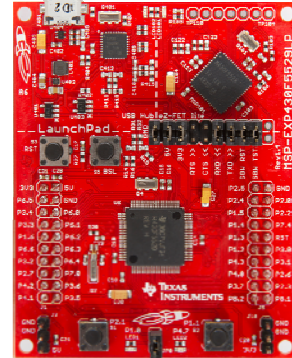
- Explore basic CCS debug functionality
Restart, Breakpoint, Single-step, Run-to-line

◆ Lab 2c – Restore Demo to Flash

- Import CCS project (for original demo)
- Load program to device's flash memory
- Verify original demo program works

◆ (Optional) Lab 2d

- Create binary TXT file of your program
- Use MSP430 Flasher to program original demo's binary file to device's flash



Time: 45 minutes

Lab Outline

Programming C with CCS	2-25
<i>Lab 2 – CCStudio Projects.....</i>	<i>2-27</i>
Lab 2a – Creating a New CCS Project	2-29
Intro to Workshop Files	2-29
Start Code Composer Studio and Open a Workspace	2-30
“CCS Edit” Perspective	2-31
Create a New Project	2-32
Build The Code (ignore advice).....	2-35
Debug The Code	2-36
Fix The Example Project	2-38
Build, Load, Connect and Run ... using the Easy Button	2-39
Lab 2b – My First Blinky.....	2-40
Create and Examine Project	2-40
Build, Load, Run.....	2-41
Restart, Single-Step, Run To Line	2-42
Lab 2c – Putting the OOB back into your device	2-44
(Optional) Lab 2d – MSP430Flasher	2-46
Programming the UE OOB demo using MSP430Flasher.....	2-46
Programming Blinky with MSP430Flasher.....	2-49
Cleanup.....	2-50

Lab 2a – Creating a New CCS Project

In this lab, you create a new CCS project that contains one source file – `hello.c` – which prints “Hello World” to the CCS console window.

The purpose of this lab is to practice creating projects and getting to know the look and feel of CCS. If you already have experience with CCS (or the Eclipse) IDE, this lab will be a quick review. The workshop labs start out very basic, but over time, they’ll get a bit more challenging and will contain less “hand holding” instructions.

Hint: In a *real-world* MSP430 program, you would **NOT want to call `printf()`**. This function is slow, requires a great deal of program and data memory, and sucks power – all bad things for any embedded application. (Real-world programs tend to replace `printf()` by sending data to a terminal via the serial port.)

We’re using this function since it’s the common starting point when working with a new processor. Part B of this lab, along with the next chapter, finds us programming what is commonly called, the “embedded” version of “hello world”. This involves blinking an LED on the target board.

Intro to Workshop Files

1. Find the workshop lab folder.

Using Windows Explorer, locate the following folder. In this folder, you will find at least two folders – aptly named for the two launchpads this workshop covers – F5529_USB, FR5969_FRAM:

```
C:\msp430_workshop\F5529_USB
C:\msp430_workshop\FR5969_FRAM
```

Click on YOUR specific target’s folder. Underneath, you’ll find many subfolders

```
C:\msp430_workshop\F5529_USB\lab_02a_ccs
C:\msp430_workshop\F5529_USB\lab_02b_blink
...
C:\msp430_workshop\F5529_USB\solutions
C:\msp430_workshop\F5529_USB\workspace
```

From this point, we will usually refer to the path using the generic **<target>** so that we can refer to whichever target board you may happen to be working with.

e.g. C:\msp430_workshop\<target>\lab_02a_ccs

So, when the instructions say “navigate to the Lab2 folder”, this assumes you are in the tree related to YOUR specific target.

Finally, you will usually work within each of the `lab_` folders but if you get stuck, you may opt to import – or examine – a lab’s archived (.zip) solution files. These are found in the `\solutions` directory.

Hint:

- This lab does not contain any “starter” files; rather, we’ll create everything from scratch.
- The readme file provides the solution code that you can copy/paste, if necessary. That said, you won’t need to do that in this lab exercise.

Start Code Composer Studio and Open a Workspace

Note: CCSv6 should already be installed; if not please refer to the workshop installation guide.

2. Start Code Composer Studio (CCS).

Double-click CCS's icon on the desktop or select it from the Windows Start menu.

3. Select a *Workspace* – don't use the default workspace location !!

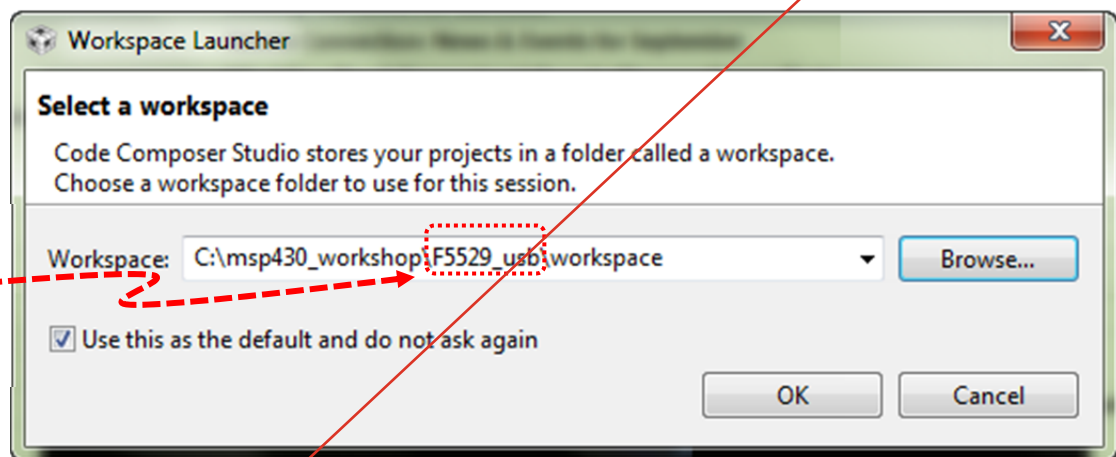
When CCS starts, a dialog box will prompt you for the location of a workspace folder. We suggest that you select the workspace folder provided in our workshop labs folder. (This will help your experience to match our lab instructions.)

Select either one of: (to match your target)

C:\msp430_workshop\F5529_usb\workspace

C:\msp430_workshop\F5529_fram\workspace

Make sure to select
FR5969
if you're using that
Launchpad



Most importantly, the workspace provides a location to store your projects ... or links to your projects. In addition to this, the workspace folder also contains many CCS preferences, such as perspectives and views. The workspace is saved automatically when CCS is closed.

Hint: If you check the “Use this as the default...” option, you won’t be asked to choose a workspace every time you open CCS. At some point, if you need to change the workspace – or create a new one – you can do this from the menu: **File → Switch Workspace**

4. Click OK to close the Select a workspace dialog.

5. After quickly examining the “Getting Started” window, you can close it, too.

When CCS opens to a new workspace, the *Getting Started* window is automatically opened and you're greeted with a variety of options. We want to mention two items:

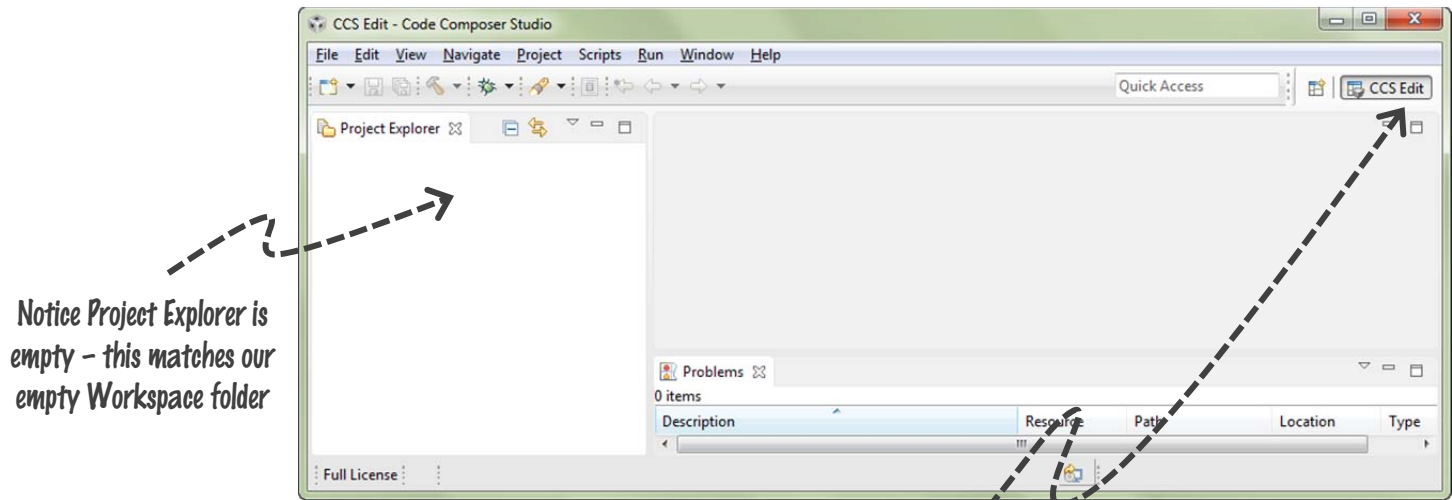
- **App Center** – you can download additional TI tools and content here. For example, this is one way to install MSP430ware or TI-RTOS.
- **Simple Mode** – We suggest that you do not put CCS into Simple Mode when following our lab instructions, as we've chosen to use the full-featured interface.

Later on, you may want to come back and check out the remaining links and videos.

“CCS Edit” Perspective

6. At this point you should see an empty CCS workbench.

The term *workbench* refers to the desktop development environment.



The workbench will open in the “CCS Edit” view.

Maximize CCS to fill your screen

Notice the tab in the upper right-hand corner...

Perspectives define the window layout views of the workbench, toolbars, and menus – as appropriate for a specific type of activity (i.e. editing or debugging). This minimizes clutter of the user interface.

- The “CCS Edit” perspective is used to when creating, editing and building C/C++ projects.
- CCS automatically switches to the “CCS Debug” perspective when a debug session is started.

You can customize the perspectives and save as many as you like.

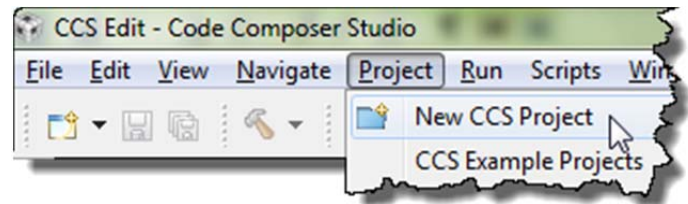
Hint: The Window → Reset Perspective... is handy for those times when you’ve changed the windows and want to get back to the original view.

Create a New Project

7. Select New CCS Project from the menu.

A *project* contains all the files you will need to develop an executable output file (.out) which can be run on the MSP430 hardware. To create a new project click:

File → New → CCS Project



8. Make project choices as shown here:

Note: Your dialog may look slightly different than this one. This is how it looked for CCSv6.0 (build 190).

a) Type “5529” or “5969” into *variant* to quickly select **Target CPU**

b) Use *Default* debugger connection (this creates the .ccxml file for you)

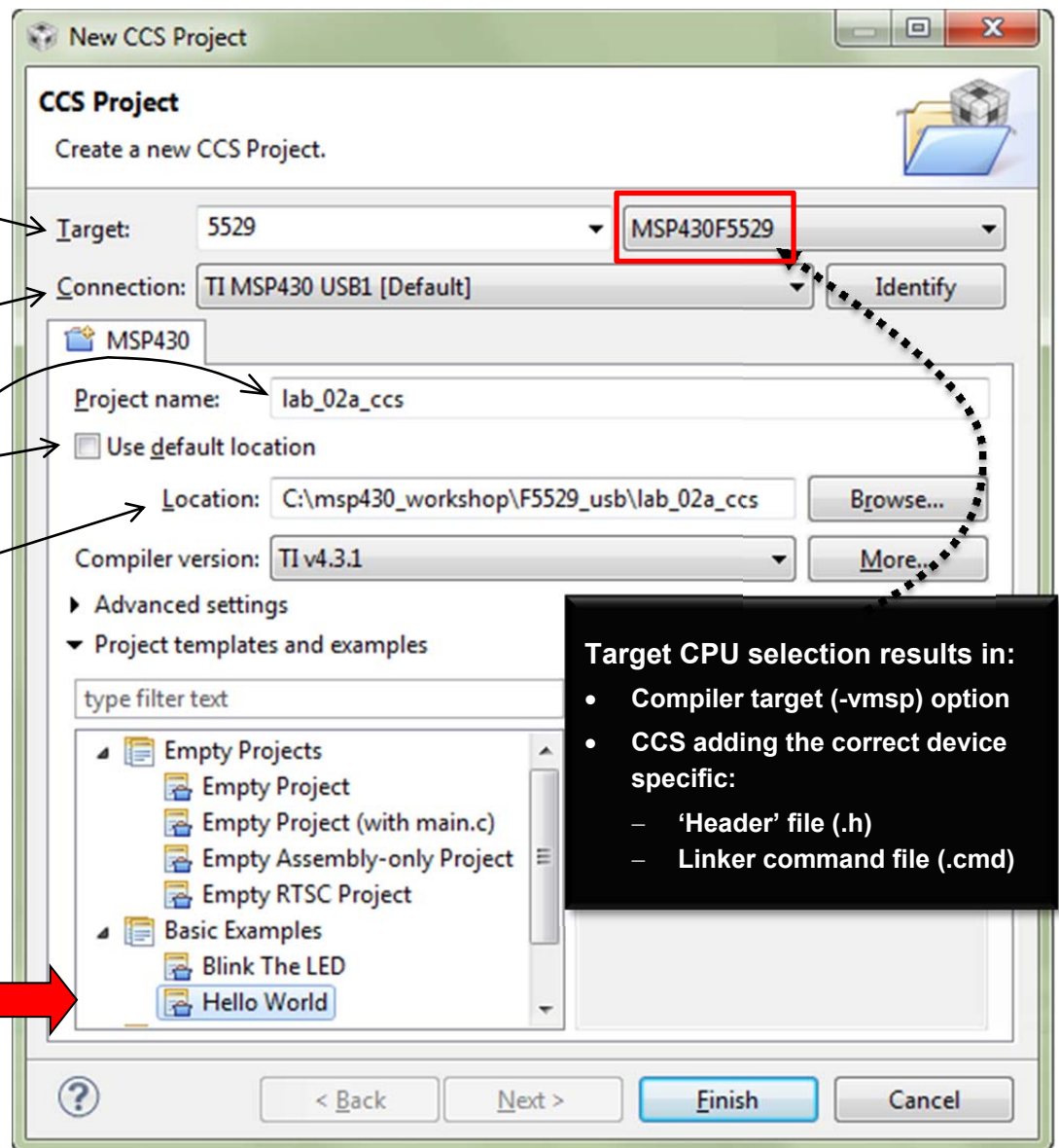
c) Name: lab_02a_ccs

d) ~~Don't use default location~~

e) ~~Choose your target's lab_02a_ccs folder~~

f) Select template: Hello World

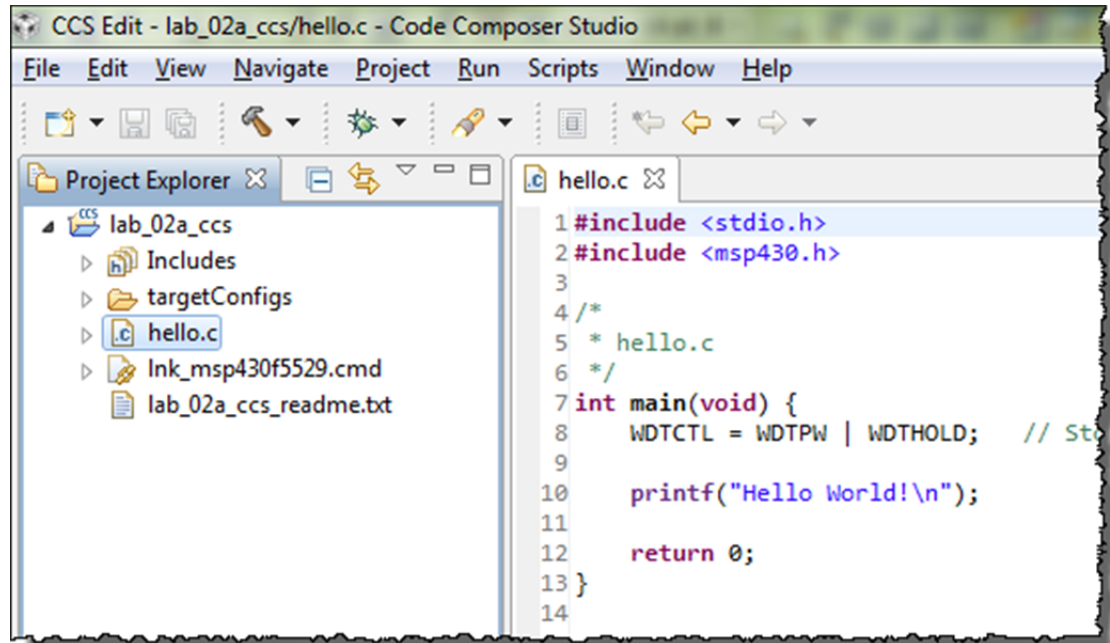
g) Click 'Finish' when done.



9. Code Composer will add the named project to your workspace.

View the project in the Project Explorer pane.

Click on the ▸ left of the project name to expand the project



CCS includes other items based upon the **Template** selection. These might include source files, libraries, etc.

When choosing the *Hello World* template, CCS adds the file hello.c to the new project.

10. Open and view lab_02a_ccs_readme.txt.

During installation, we placed the readme file into the project folder.

By default, Eclipse (and thus CCS) adds any file it finds within the project folder to the project. This is why the readme text file shows up in project explorer. Go ahead and open it up:

Double-click on: lab_02a_ccs_readme.txt

You should see a description of this lab similar to the abstract found in these lab directions.

Hint: Be aware of this Eclipse feature. If – say in Windows Explorer – you absent-mindedly add a C source file to your project folder, it will become part of your program the next time you build.

If you want a file in the project folder, but not in your program, you can exclude files from build:

Right-click on the file → Exclude from Build

11. Explore source code in hello.c.

Open the file, if it's not already open.

Double-click on hello.c in the Project Explorer window

We hope most of this code is self-explanatory. Except for one line, it's all standard C code:

```
#include <stdio.h>
#include <msp430.h>

/*
 * hello.c
 */
int main(void) {
    WDTCTL = WDTPW | WDTHOLD;    // Stop watchdog timer
    printf("Hello World!\n");
    return 0;
}
```

The only MSP430-specific line is the same one we examined in the chapter discussion:

```
WDTCTL = WDTPW | WDTHOLD;    // Stop watchdog timer
```

As the comment indicates, this turns off the watchdog timer (WDT peripheral). As we'll learn in Chapter 4, the WDT peripheral is always turned on (by default) in MSP430 devices. If we don't turn it off, it will reset the system – which is not what we usually want during development (especially during 'hello world').

Build The Code (ignore advice)

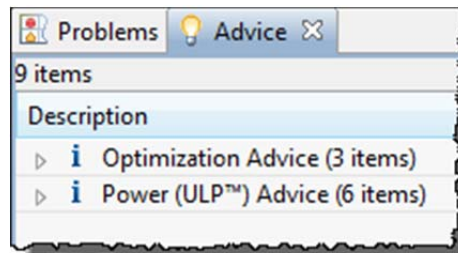
12. Build your project using “the hammer” and check for errors.

At this point, it is a good time to build your code to check for any errors before moving on.

Just click the “hammer” icon:



It should build without any *Problems*, although you should see two sets of Advice: Optimization Advice (new to CCSv5.5) and Power (ULP™) Advice.

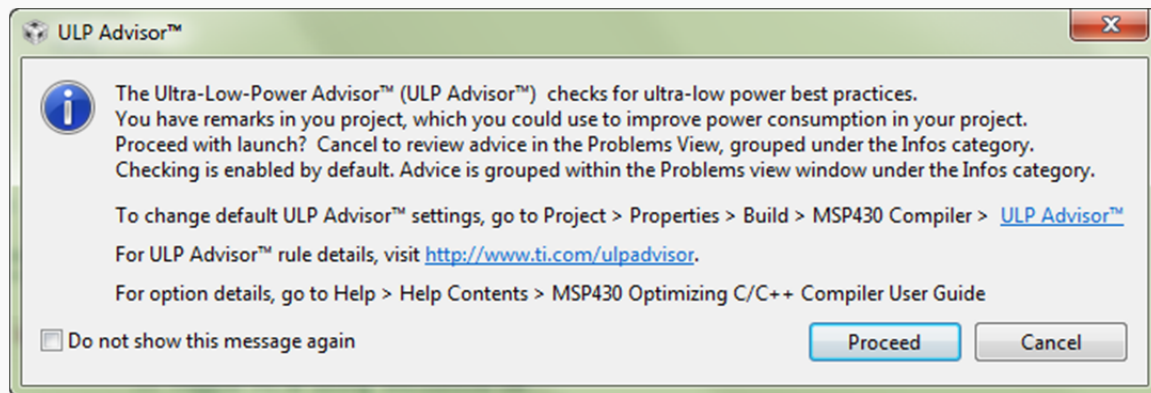


At this point, we’re just going to ignore their advice. It’s better to get code running first. Later, we return and investigate some of these items further.

If the program builds successfully, move to the next page to begin debugging. If you have problems getting it to build, please ask a neighbor, or your instructor for help.

Sidenote: ULP Advisor

Sometime, when you launch the debugger (as we will soon), CCS will warn you that your code could be better optimized for lower power.



While we like the ULP Advisor tool, this usually comes up a long time before we are ready to start optimizing our performance. We recommend that you click the box:

☒ Do not show this message again

As the dialog above indicates, you can always go into your project’s properties and enable or disable this advice. We will do this in a later chapter, when we’re ready to focus on driving our every last Nano amp.

Debug The Code

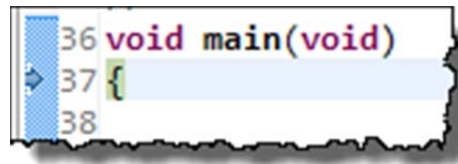
13. Debug your program.

Clicking the Debug button will: Build the program (if needed); Launch the debugger; Connect to Target; and Load your program

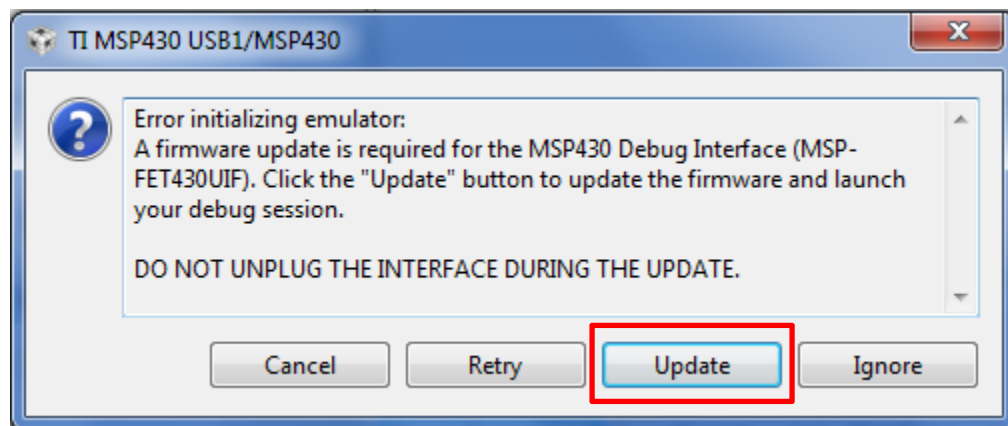
Click the BUG toolbar button:



Your program will now download to the target board and the PC will automatically run until it reaches `main()`, then stop as shown:



Note: The first time you Launch a debugger session, you may encounter the following dialog:



This occurs when CCS finds that the FET firmware – that is, the firmware in your Launchpad’s debugger – is out-of-date. We recommend that you choose to update the firmware. Once complete, CCS should finish launching the debugger.

Connection Problems - Troubleshooting

If the error “cannot connect to target” appears, the problem is most likely due to:

- No target configuration (.ccxml) file
- Wrong board/target config file or both – i.e. board does not match the target config file
- Bad USB cable
- Windows USB driver is incorrect – or just didn’t get enumerated correctly

If you run into this, check for each of these possibilities. In the case of the Windows USB driver try:

- Unplugging the USB cable and trying it in a different USB port. (Just changing ports can often get Windows to re-enumerate the device.
- Open Windows Device Manager and verify the board exists and there are no warnings or errors with its driver.
- If all else fails, ask your neighbor (or instructor) for assistance.

14. Run the code.

Now, it's finally time to RUN or "Play". ► Hit the **Resume** button:



The button is called 'Resume', though we may end up calling it 'Play' since that's what the icon looks like.

15. Pause the code.

To stop your program running, ► click the **Suspend** button to pause):

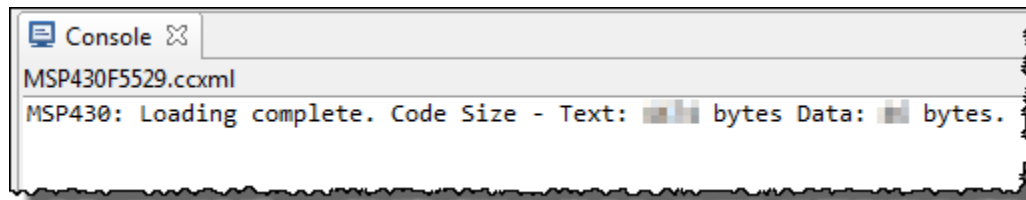


Warning: Suspend is different than Terminate !!!

If you click the Terminate button, the debugger – and your connection to the target – will be closed. If you're debugging and just want to view a variable or memory, you will have to open a new debug session all over again. Remember to *pause* and think, before you halting your program.

16. Did printf work?

Did "Hello World!" show up in your console window?



Nope, it didn't show up for us. ☹

17. Let's Terminate the debug session and go fix "their" project.

This time we really want to terminate our debug session.

Click the red Terminate button:



This closes the debug session (and Debug Perspective). CCS will switch back to the *Edit* perspective. You are now completely disconnected from the target.

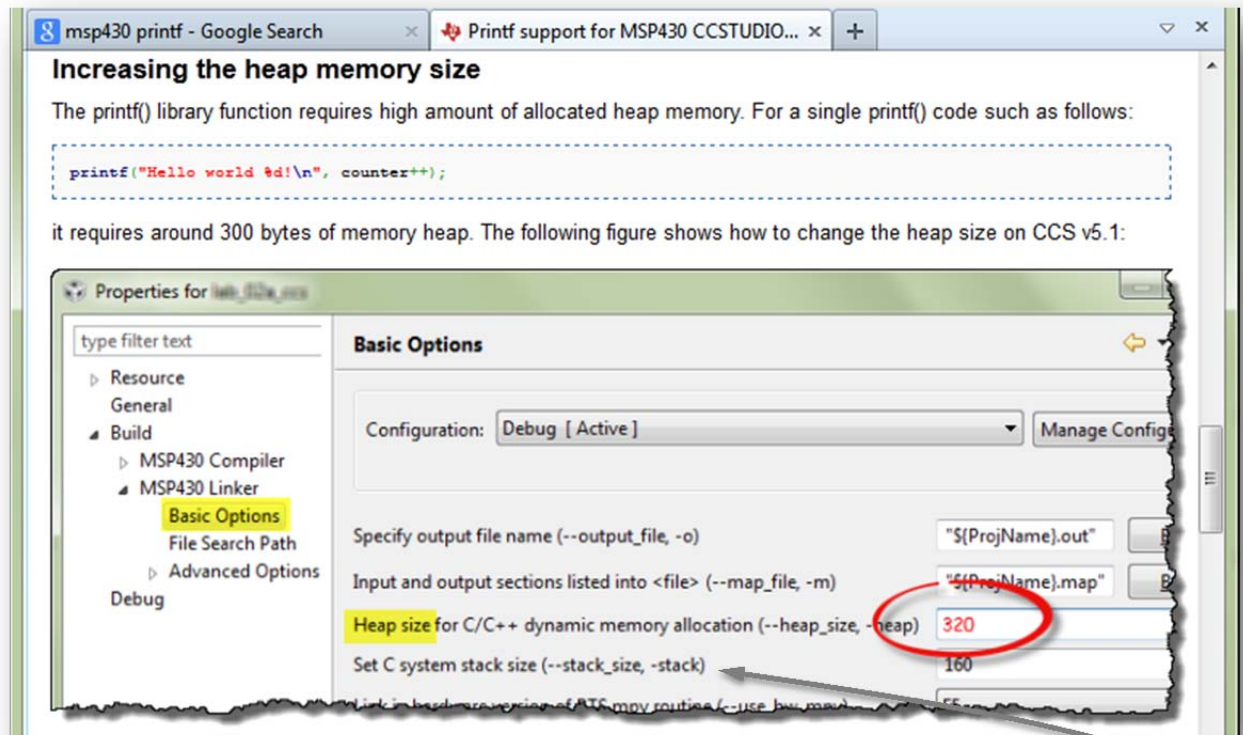
Fix The Example Project

18. What is wrong? Increase the heap size.

Per the wiki suggestion, let's increase the heap size to 320 bytes.

Right-click project → Properties → MSP430 Linker → Basic Options

Increase *Heap size* to: **320**



You can find a description of this problem by searching the internet for: “msp430 printf”

From that, you should find a MSP430 wiki page that describes how to get printf() to work:

http://processors.wiki.ti.com/index.php/Printf_support_for_MSP430_CCSTUDIO_compiler

(In fact, this is how we figured out how to solve the problem.)

Hint: As a side note, if you look just below the entry for setting the Heap size, you will see the setting for Stack size. This is where you would change the stack size of you system, if you ever need to do that.

Build, Load, Connect and Run ... using the Easy Button

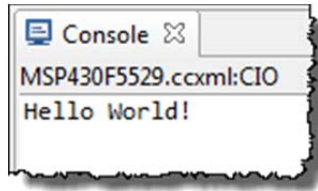


19. Rebuild and Reload your program.

First, make sure you terminated your previous debug session and you are in the Edit perspective.



20. Once the program has successfully loaded, ► run it.



21. Terminate and Close the lab_02a_ccs project.

Terminate the debug session and then close the project. Closing a project is both handy and prevents errors.

Right-click project → Close Project

If your source file (hello.c) was open, notice how closing the project also closes most source files. This can help prevent errors. *(Wait until you've spent an hour editing a file – with it not working – only to find you were editing a file with the same name, but from a different project. Doh!)*

You can quickly reopen the project, when and if you need to.

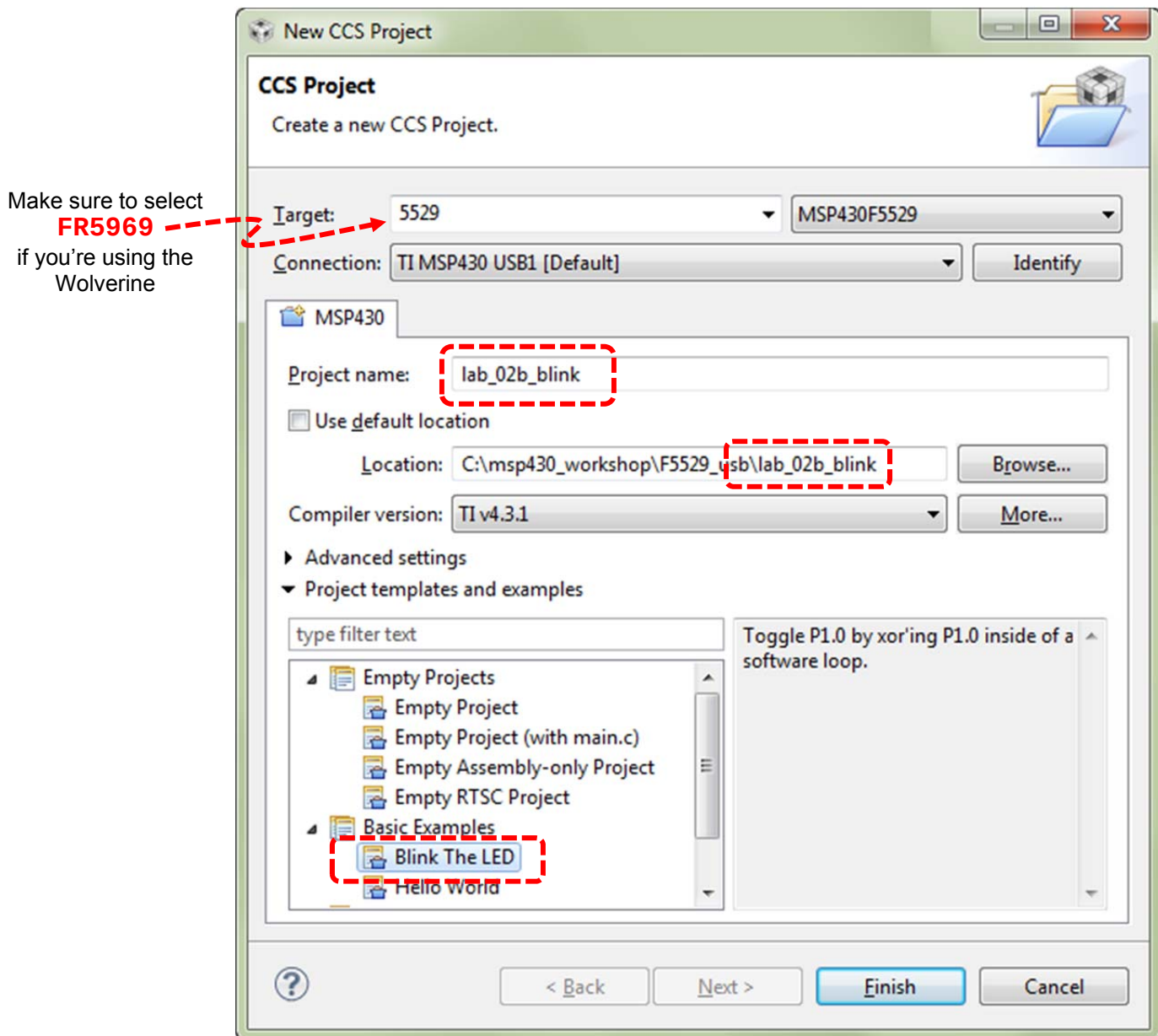
Lab 2b – My First Blinky

We plan to get into all the details of how GPIO (general purpose input/output) works in the next chapter. At that time, we will also introduce the MSP430ware DriverLib library to help you program GPIO, as well as all the other peripherals on the MSP430.

In the lab exercise, we want to teach you a few additional debugging basics – and need some code to work with. To that end, we’re going to use the Blink template found in CCS. This is generic, low-level MSP430 code, but it should suite our purposes for now.

Create and Examine Project

1. Create a new project (lab_02b_blink) with the following properties:



2. Let's quickly examine the code that was in the template.

This code simply blinks the LED connected to Port1, Pin0 (often shortened to P1.0).

```
#include <msp430.h>

int main(void) {
    WDTCTL = WDTPW | WDTHOLD;    // Stop watchdog timer

    P1DIR |= 0x01;                // Set P1.0 to out-put direction

    for(;;) {
        volatile unsigned int i; // volatile to prevent optimization

        P1OUT ^= 0x01;           // Toggle P1.0 using exclusive-OR

        i = 10000;               // SW Delay
        do i--;
        while(i != 0);
    }
}
```

Other than standard C code which creates an endless loop that repeats every 10,000 counts, there are three MSP430-specific lines of code.

- As we saw earlier, the Watchdog Timer needs to be halted.
- The I/O pin (P1.0) needs to be configured as an output. This is done by writing a “1” to bit 0 of the Port1 direction register (P1DIR).
- Finally, each time thru the for loop, the code toggles the value of the P1.0 pin.
(In this case, it appears the author didn't really care if his LED started in the on or off position; just that it changed each time thru the loop.)

Hint: As we mentioned earlier, we will provide more details about the MSP430 GPIO features, registers, and programming in the next chapter.

Build, Load, Run



3. Build the code. Start the debugger. Load the code.

If you don't remember how, please refer back to *lab_02a_ccs*.



4. Let's start by just running the code.

Click the **Resume** button on the toolbar (or press **F8**)

You should see the LED toggling on/off.



5. Halt the debugger by clicking the “Suspend” button ... don't terminate!

Restart, Single-Step, Run To Line

6. Restart your program.

Let's get the program counter back to the beginning of our program.

Run → Restart - or - use the Restart toolbar button:



Notice how the arrow, which represents the Program Counter (PC) ends up at `main()` after your restart your program. This is where your code will start executing next.

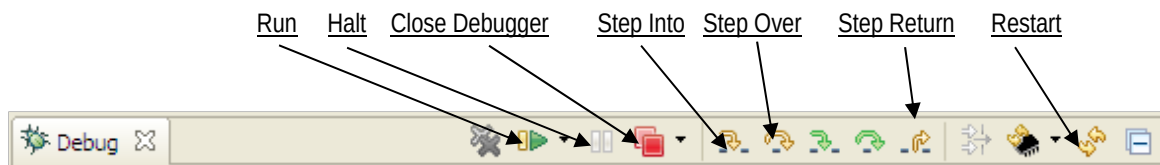
In CCS, the default is for execution to stop whenever it reaches the `main()` routine.

By the way, **Restart** starts running your code from the entry point specified in the executable (.out) file. Most often, this is set to your reset vector. On the other hand, **Reset** will invoke an actual reset. (*Reset will be discussed further in Chapter 4.*)

```
21 #include <msp430.h>
22
23 int main(void) {
24     WDTCTL = WDTPW | WDTHOLD;
25     P1DIR |= 0x01;
26 }
```

7. Single-step your program.

With the program halted, click the **Step Over (F6)** toolbar button (or tap the F6 key):



Notice how one line of code is executed each time you click *Step Over*; in fact, this action treats functions calls as a single point of execution – that is, it steps *over* them. On the other hand *Step Into* will execute a function call step-by-step – go *into* it. Step Return helps to jump back out of any function call you're executing.

Hint: You probably won't see anything happen until you have stepped past the line of code that toggles P1.0.

8. Single-step 10,000 times

Try stepping over-and-over again until the light toggles again...

Hmmm... looking at the count of 10,000; we could be single-stepping for a long time. For this, we have something better...

9. Try the *Run-To-Line* feature.

Click on the line of code that toggles the LED.

Click on the line: `P1OUT ^= 0x01;`

Then Right-click and select **Run To Line** (or hit Ctrl-R)

Single-step once more to toggle the LED

10. Set a breakpoint.

There are many ways to set a breakpoint on a line of code in CCS. You can right-click on a line of code to toggle a Breakpoint. But the easiest is to:

Double-click the blue bar on the line of code

For example, you can see we have just set a breakpoint on our toggle LED line of code:

Once a breakpoint is set, there will be a blue marker that represents it. By **double-clicking** in this location, we can easily add or remove breakpoints.

**11. Run to breakpoint.**

Run the code again. Notice how it stops at the breakpoint each time the program flow encounters it.

Press F8 (multiple times)

You should see the LED toggling on or off each time you run the code.

12. Terminate your debug session.

When you're done having fun, terminate your debug session.

13. Close the project.

If any edit windows are still open after closing the project, we recommend closing them, too.

Note: When using early versions of CCSv6 with the 'FR5969 device, under some circumstances, CCS may corrupt your program in Flash memory if you have more than one breakpoint set. This usually occurs when restarting or resetting your program during debug. The easiest way to visualize this is to view your main() function using the *Disassembly Window*.

The workarounds include:

1. Clear all breakpoints before resetting, restarting or terminating your program.
 2. Load a different program; then load the program that has become corrupted.
-